Tes Teknis: Backend Engineer Transjakarta - Sistem Manajemen Armada

Deskripsi Proyek

Anda diminta untuk mengembangkan layanan backend untuk sistem manajemen armada yang dapat:

1. **Menerima data lokasi kendaraan melalui MQTT**.
2. **Menyimpan data lokasi ke PostgreSQL**.
3. **Menyediakan API untuk mendapatkan lokasi terakhir dan riwayat perjalanan kendaraan**.
4. **Menggunakan RabbitMQ untuk memicu event ketika kendaraan memasuki area tertentu (geofence)**.
5. **Menggunakan Docker untuk menjalankan seluruh sistem**.

Teknologi yang Digunakan

- **Golang** untuk pengembangan layanan backend.
- **MQTT (Eclipse Mosquitto)** untuk menerima data lokasi kendaraan.
- **PostgreSQL** untuk menyimpan data lokasi kendaraan.
- **RabbitMQ** untuk memproses event berbasis geofence.
- **Docker** untuk menjalankan seluruh sistem dalam container.

Spesifikasi Teknis

1. Menerima Data Lokasi Kendaraan via MQTT

Buat **subscriber MQTT** di Golang yang mendengarkan topik berikut:
/fleet/vehicle/{vehicle_id}/location

-

Data lokasi dikirim dalam format **JSON** seperti berikut:

```
{
 "vehicle_id": "B1234XYZ",
 "latitude": -6.2088,
```

```
  "longitude": 106.8456,
  "timestamp": 1715003456
}
```

- Pastikan validasi dilakukan untuk memastikan data yang diterima sesuai format.

2. Menyimpan Data Lokasi ke PostgreSQL

Simpan data lokasi kendaraan dalam tabel berikut:
Table Name:vehicle_loctions
Field: vehicle_id, latitude, longitude, timestamp

- Implementasikan **service di Golang** yang memasukkan data ke PostgreSQL.

3. Menyediakan API untuk Mengakses Data Lokasi

- Buat **REST API** menggunakan **Gin/Fiber** dengan endpoint berikut:

Mendapatkan lokasi terakhir kendaraan
GET /vehicles/{vehicle_id}/location
Contoh respons:

```
 {
  "vehicle_id": "B1234XYZ",
  "latitude": -6.2088,
  "longitude": 106.8456,
  "timestamp": 1715003456
}
```

Mendapatkan riwayat lokasi dalam rentang waktu tertentu
GET /vehicles/{vehicle_id}/history?start=1715000000&end=1715009999

4. Menggunakan RabbitMQ untuk Event Geofence

- Jika kendaraan masuk dalam radius **50 meter** dari titik tertentu, event harus dikirim ke **RabbitMQ**.
- Konfigurasi:
  - **Exchange:** `fleet.events`
  - **Queue:** `geofence_alerts`

Format pesan yang dikirim ke RabbitMQ:

```
 {
  "vehicle_id": "B1234XYZ",
  "event": "geofence_entry",
  "location": {
    "latitude": -6.2088,
```

```
    "longitude": 106.8456
  },
  "timestamp": 1715003456
}
```

- Buat **worker service** yang membaca data dari `geofence_alerts` queue.

5. Docker & Deployment

- Buat **Docker Compose** yang mencakup:
  - Layanan backend Golang
  - PostgreSQL
  - RabbitMQ
  - MQTT broker (Eclipse Mosquitto)
- Buat **Dockerfile** untuk aplikasi Golang.

Persyaratan Tambahan

Kandidat diwajibkan untuk membuat script yang mempublikasikan data lokasi kendaraan ke MQTT.

- Script bisa dibuat dalam **Go**.
- Script harus mengirim data mock ke topik MQTT `/fleet/vehicle/{vehicle_id}/location` setiap 2 detik.

Cara Pengujian

Pengujian	Hasil yang Diharapkan
Publikasi data MQTT	Data lokasi kendaraan berhasil dikirim ke MQTT.
Penyimpanan PostgreSQL	Data yang diterima disimpan di database dengan benar.
API Lokasi Terakhir	Endpoint `GET /vehicles/{vehicle_id}/location` mengembalikan data yang benar.
API Riwayat Lokasi	Endpoint `GET /vehicles/{vehicle_id}/history` mengembalikan data dalam rentang waktu tertentu.

RabbitMQ Geofence Event	Pesan dikirim ke RabbitMQ saat kendaraan memasuki geofence.
Docker Compose	Semua layanan berjalan dengan lancar.

Cara Penyerahan

1. Upload kode ke **GitHub repository**.
2. Sertakan **README.md** berisi cara menjalankan aplikasi.
3. Berikan **Postman Collection** untuk menguji API.
4. Pastikan **Docker Compose** dapat menjalankan seluruh sistem.

Selain mengunggah kode ke GitHub beserta README.md dan Postman Collection, kandidat juga diwajibkan untuk:

- Membuat **video demo** berdurasi 5–10 menit.
- Video harus menjelaskan:
    1. Arsitektur singkat sistem (bagaimana tiap komponen—MQTT, PostgreSQL, RabbitMQ, Docker—berinteraksi).
    2. Menunjukkan aplikasi dijalankan menggunakan `docker compose up`.
    3. Menunjukkan publikasi data mock ke MQTT dan bagaimana data itu masuk ke PostgreSQL.
       Mengakses API:
        - `GET /vehicles/{vehicle_id}/location`
        - `GET /vehicles/{vehicle_id}/history?start=...&end=...`
    4. Menunjukkan event geofence terkirim ke RabbitMQ (`geofence_alerts` queue).
- Video harus memperlihatkan sistem berjalan **100% sesuai spesifikasi**.
